

Diseño de Base de Datos: DataForge IA

1. Introducción

El presente documento detalla el diseño de la base de datos para el sistema **DataForge IA**, una plataforma web de análisis predictivo para PYMES. El objetivo es establecer un esquema de base de datos relacional que garantice la integridad, consistencia y eficiencia en el almacenamiento y recuperación de la información. Este diseño considera las entidades principales del sistema, sus atributos, las relaciones entre ellas, la definición de llaves primarias y foráneas, y los principios básicos de normalización para optimizar el rendimiento y la mantenibilidad.

2. Entidades y Atributos

Las siguientes entidades representan los objetos de información clave en el sistema DataForge IA:

2.1. Cuentas

Representa la organización (PYME) que utiliza la plataforma.

- `cuenta_id` (PK, UUID): Identificador único de la cuenta.
- `nombre_empresa` (VARCHAR(255)): Nombre de la empresa.
- `plan_suscripcion` (VARCHAR(50)): Tipo de plan de suscripción (ej. 'Basico', 'Pro', 'Enterprise').
- `fecha_creacion` (TIMESTAMP): Fecha y hora de creación de la cuenta.
- `estado` (VARCHAR(50)): Estado actual de la cuenta (ej. 'Activa', 'Suspendida', 'Cancelada').
- `uso_almacenamiento_gb` (DECIMAL(10,2)): Almacenamiento utilizado en GB.

2.2. Usuarios

Representa a los individuos que acceden a la plataforma dentro de una `Cuenta`.

- `usuario_id` (PK, UUID): Identificador único del usuario.
- `cuenta_id` (FK, UUID): Referencia a la cuenta a la que pertenece el usuario.
- `nombre` (VARCHAR(255)): Nombre completo del usuario.

- `email` (VARCHAR(255), UNIQUE): Correo electrónico del usuario, utilizado para el inicio de sesión.
- `password_hash` (VARCHAR(255)): Hash de la contraseña del usuario.
- `rol` (VARCHAR(50)): Rol del usuario dentro de la cuenta (ej. 'Administrador', 'Analista', 'Lector').
- `fecha_registro` (TIMESTAMP): Fecha y hora de registro del usuario.
- `ultimo_acceso` (TIMESTAMP): Última fecha y hora de acceso del usuario.
- `estado` (VARCHAR(50)): Estado de la cuenta de usuario (ej. 'Activo', 'Inactivo', 'Pendiente_Verificacion').

2.3. ConjuntosDeDatos

Representa los archivos de datos cargados por los usuarios.

- `conjunto_datos_id` (PK, UUID): Identificador único del conjunto de datos.
- `usuario_id` (FK, UUID): Referencia al usuario que cargó el conjunto de datos.
- `cuenta_id` (FK, UUID): Referencia a la cuenta propietaria del conjunto de datos.
- `nombre` (VARCHAR(255)): Nombre asignado al conjunto de datos.
- `formato` (VARCHAR(10)): Formato del archivo (ej. 'CSV', 'XLSX').
- `tamaño_bytes` (BIGINT): Tamaño del archivo en bytes.
- `ruta_almacenamiento` (VARCHAR(512)): Ruta o URL al archivo en el almacenamiento de objetos.
- `fecha_carga` (TIMESTAMP): Fecha y hora de carga del archivo.
- `estado_procesamiento` (VARCHAR(50)): Estado del procesamiento (ej. 'Pendiente', 'Procesado', 'Error').

2.4. EsquemasDeDatos

Define la estructura y el mapeo de las columnas de un `ConjuntoDeDatos`.

- `esquema_id` (PK, UUID): Identificador único del esquema de datos.
- `conjunto_datos_id` (FK, UUID, UNIQUE): Referencia al conjunto de datos al que pertenece este esquema.
- `fecha_creacion` (TIMESTAMP): Fecha y hora de creación del esquema.

2.5. ColumnasDeEsquema

Representa una columna individual dentro de un EsquemaDeDatos .

- `columna_id` (PK, UUID): Identificador único de la columna.
- `esquema_id` (FK, UUID): Referencia al esquema de datos al que pertenece la columna.
- `nombre_original` (VARCHAR(255)): Nombre de la columna en el archivo original.
- `nombre_mapeado` (VARCHAR(255)): Nombre de la columna después del mapeo (estandarizado).
- `tipo_dato` (VARCHAR(50)): Tipo de dato asignado (ej. 'Numerico', 'Categorico', 'Fecha', 'Texto').
- `es_objetivo` (BOOLEAN): Indica si esta columna es la variable objetivo para la predicción.

2.6. ModelosPredictivos

Almacena la configuración y metadatos de los modelos de IA.

- `modelo_id` (PK, UUID): Identificador único del modelo predictivo.
- `usuario_id` (FK, UUID): Referencia al usuario que creó/configuró el modelo.
- `cuenta_id` (FK, UUID): Referencia a la cuenta propietaria del modelo.
- `nombre` (VARCHAR(255)): Nombre del modelo.
- `descripcion` (TEXT): Descripción del modelo.
- `tipo_modelo` (VARCHAR(50)): Tipo de algoritmo (ej. 'Ventas', 'Inventario', 'RotacionClientes').
- `parametros_config` (JSONB): Parámetros de configuración específicos del modelo (ej. horizonte de tiempo, frecuencia).
- `estado` (VARCHAR(50)): Estado del modelo (ej. 'NoEntrenado', 'Entrenando', 'Listo', 'Error').
- `fecha_creacion` (TIMESTAMP): Fecha y hora de creación del modelo.
- `fecha_ultimo_entrenamiento` (TIMESTAMP): Última fecha y hora de entrenamiento exitoso.

2.7. Predicciones

Almacena los resultados generados por un ModeloPredictivo .

- `prediccion_id` (PK, UUID): Identificador único de la predicción.

- `modelo_id` (FK, UUID): Referencia al modelo que generó esta predicción.
- `conjunto_datos_id` (FK, UUID): Referencia al conjunto de datos utilizado para generar la predicción.
- `fecha_generacion` (TIMESTAMP): Fecha y hora en que se generó la predicción.
- `resultados_raw` (JSONB): Resultados de la predicción en formato JSON (ej. valores predichos, intervalos de confianza).
- `metricas_precision` (JSONB): Métricas de evaluación del modelo (ej. MAE, RMSE, R2).

2.8. Reportes

Almacena la configuración de los reportes y paneles de control.

- `reporte_id` (PK, UUID): Identificador único del reporte.
- `usuario_id` (FK, UUID): Referencia al usuario que creó el reporte.
- `cuenta_id` (FK, UUID): Referencia a la cuenta propietaria del reporte.
- `nombre` (VARCHAR(255)): Título del reporte.
- `tipo_grafico` (VARCHAR(50)): Tipo de visualización principal (ej. 'Lineas', 'Barras', 'Pastel').
- `configuracion_visual` (JSONB): Configuración detallada de la visualización (filtros, ejes, colores).
- `fecha_creacion` (TIMESTAMP): Fecha y hora de creación del reporte.
- `fecha_ultima_modificacion` (TIMESTAMP): Última fecha y hora de modificación.

2.9. Alertas

Almacena la configuración de las alertas para las predicciones.

- `alerta_id` (PK, UUID): Identificador único de la alerta.
- `usuario_id` (FK, UUID): Referencia al usuario que configuró la alerta.
- `cuenta_id` (FK, UUID): Referencia a la cuenta propietaria de la alerta.
- `modelo_id` (FK, UUID): Referencia al modelo predictivo asociado a la alerta.
- `metrica_objetivo` (VARCHAR(255)): Métrica o columna que se monitorea (ej. 'ventas_predichas').
- `condicion` (VARCHAR(50)): Condición de la alerta (ej. 'MayorQue', 'MenorQue', 'Igual').
- `umbral` (DECIMAL(18,2)): Valor umbral que dispara la alerta.

- `activa` (BOOLEAN): Indica si la alerta está activa o inactiva.
- `fecha_creacion` (TIMESTAMP): Fecha y hora de creación de la alerta.
- `ultima_notificacion` (TIMESTAMP): Última fecha y hora en que se envió una notificación.

3. Relaciones y Llaves

Las relaciones entre las entidades se establecen mediante llaves foráneas (FK) que referencian a las llaves primarias (PK) de otras tablas. A continuación, se describen las relaciones clave:

- **Cuentas y Usuarios**: Una `Cuenta` puede tener muchos `Usuarios`. La tabla `Usuarios` tiene una FK `cuenta_id` que referencia a `Cuentas.cuenta_id`.
- **Usuarios y ConjuntosDeDatos**: Un `Usuario` puede cargar muchos `ConjuntosDeDatos`. La tabla `ConjuntosDeDatos` tiene una FK `usuario_id` que referencia a `Usuarios.usuario_id`.
- **Cuentas y ConjuntosDeDatos**: Un `ConjuntoDeDatos` pertenece a una `Cuenta`. La tabla `ConjuntosDeDatos` tiene una FK `cuenta_id` que referencia a `Cuentas.cuenta_id`.
- **ConjuntosDeDatos y EsquemasDeDatos**: Un `ConjuntoDeDatos` tiene un único `EsquemaDeDatos`. La tabla `EsquemasDeDatos` tiene una FK `conjunto_datos_id` que referencia a `ConjuntosDeDatos.conjunto_datos_id` y es UNIQUE.
- **EsquemasDeDatos y ColumnasDeEsquema**: Un `EsquemaDeDatos` tiene muchas `ColumnasDeEsquema`. La tabla `ColumnasDeEsquema` tiene una FK `esquema_id` que referencia a `EsquemasDeDatos.esquema_id`.
- **Usuarios y ModelosPredictivos**: Un `Usuario` puede crear muchos `ModelosPredictivos`. La tabla `ModelosPredictivos` tiene una FK `usuario_id` que referencia a `Usuarios.usuario_id`.
- **Cuentas y ModelosPredictivos**: Un `ModeloPredictivo` pertenece a una `Cuenta`. La tabla `ModelosPredictivos` tiene una FK `cuenta_id` que referencia a `Cuentas.cuenta_id`.
- **ModelosPredictivos y Predicciones**: Un `ModeloPredictivo` puede generar muchas `Predicciones`. La tabla `Predicciones` tiene una FK `modelo_id` que referencia a `ModelosPredictivos.modelo_id`.
- **ConjuntosDeDatos y Predicciones**: Una `Prediccion` se basa en un `ConjuntoDeDatos`. La tabla `Predicciones` tiene una FK `conjunto_datos_id` que referencia a `ConjuntosDeDatos.conjunto_datos_id`.

- **Usuarios y Reportes** : Un Usuario puede crear muchos Reportes . La tabla Reportes tiene una FK usuario_id que referencia a Usuarios.usuario_id .
- **Cuentas y Reportes** : Un Reporte pertenece a una Cuenta . La tabla Reportes tiene una FK cuenta_id que referencia a Cuentas.cuenta_id .
- **Usuarios y Alertas** : Un Usuario puede configurar muchas Alertas . La tabla Alertas tiene una FK usuario_id que referencia a Usuarios.usuario_id .
- **Cuentas y Alertas** : Una Alerta pertenece a una Cuenta . La tabla Alertas tiene una FK cuenta_id que referencia a Cuentas.cuenta_id .
- **ModelosPredictivos y Alertas** : Una Alerta está asociada a un ModeloPredictivo . La tabla Alertas tiene una FK modelo_id que referencia a ModelosPredictivos.modelo_id .

4. Normalización Básica

El diseño propuesto sigue los principios de normalización de bases de datos para reducir la redundancia de datos y mejorar la integridad. Se busca alcanzar al menos la Tercera Forma Normal (3NF).

- **Primera Forma Normal (1NF)**: Todas las tablas tienen una clave primaria y todos los atributos son atómicos (no hay grupos repetitivos ni atributos multivaluados).
- **Segunda Forma Normal (2NF)**: Todas las tablas están en 1NF y todos los atributos no clave dependen completamente de la clave primaria completa. Esto se asegura al tener claves primarias simples o compuestas que identifican de forma única cada fila.
- **Tercera Forma Normal (3NF)**: Todas las tablas están en 2NF y no hay dependencias transitivas de atributos no clave sobre la clave primaria. Por ejemplo, la información de la cuenta se almacena solo en la tabla Cuentas y se referencia desde Usuarios mediante una FK, evitando duplicar datos de la cuenta en la tabla de usuarios.

El uso de UUIDs como llaves primarias proporciona unicidad global y facilita la distribución de la base de datos en entornos de microservicios. Los tipos de datos JSONB se utilizan para almacenar configuraciones flexibles (ej. parametros_config en ModelosPredictivos , configuracion_visual en Reportes), lo que permite flexibilidad sin sacrificar la estructura relacional para los datos principales.

5. Diagrama Entidad-Relación (DER) (Explicado Textualmente)

El Diagrama Entidad-Relación (DER) ilustra visualmente las entidades descritas y sus interconexiones. Se representarán las tablas como rectángulos, los atributos como listas dentro

de los rectángulos (con PK y FK indicadas), y las relaciones como líneas que conectan las tablas, mostrando la cardinalidad (ej. 1 a muchos, 1 a 1).

El DER incluirá:

- **Cuentas** : Con `cuenta_id` como PK.
- **Usuarios** : Con `usuario_id` como PK y `cuenta_id` como FK a **Cuentas** .
- **ConjuntosDeDatos** : Con `conjunto_datos_id` como PK, `usuario_id` como FK a **Usuarios** y `cuenta_id` como FK a **Cuentas** .
- **EsquemasDeDatos** : Con `esquema_id` como PK y `conjunto_datos_id` como FK a **ConjuntosDeDatos** (UNIQUE).
- **ColumnasDeEsquema** : Con `columna_id` como PK y `esquema_id` como FK a **EsquemasDeDatos** .
- **ModelosPredictivos** : Con `modelo_id` como PK, `usuario_id` como FK a **Usuarios** y `cuenta_id` como FK a **Cuentas** .
- **Predicciones** : Con `prediccion_id` como PK, `modelo_id` como FK a **ModelosPredictivos** y `conjunto_datos_id` como FK a **ConjuntosDeDatos** .
- **Reportes** : Con `reporte_id` como PK, `usuario_id` como FK a **Usuarios** y `cuenta_id` como FK a **Cuentas** .
- **Alertas** : Con `alerta_id` como PK, `usuario_id` como FK a **Usuarios** , `cuenta_id` como FK a **Cuentas** y `modelo_id` como FK a **ModelosPredictivos** .

Las relaciones se visualizarán con la cardinalidad apropiada, por ejemplo, una cuenta tiene muchos usuarios, un usuario carga muchos conjuntos de datos, etc.

6. Código PlantUML para el Diagrama Entidad-Relación (DER)

```
@startuml
skinparam linetype ortho

entity Cuentas {
    * cuenta_id: UUID <<PK>>
    --
    nombre_empresa: VARCHAR(255)
    plan_suscripcion: VARCHAR(50)
    fecha_creacion: TIMESTAMP
    estado: VARCHAR(50)
    uso_almacenamiento_gb: DECIMAL(10,2)
}

entity Usuarios {
    * usuario_id: UUID <<PK>>
    --
    # cuenta_id: UUID <<FK>>
    nombre: VARCHAR(255)
    email: VARCHAR(255) <<UNIQUE>>
    password_hash: VARCHAR(255)
    rol: VARCHAR(50)
    fecha_registro: TIMESTAMP
    ultimo_acceso: TIMESTAMP
    estado: VARCHAR(50)
}

entity ConjuntosDeDatos {
    * conjunto_datos_id: UUID <<PK>>
    --
    # usuario_id: UUID <<FK>>
    # cuenta_id: UUID <<FK>>
    nombre: VARCHAR(255)
    formato: VARCHAR(10)
    tamaño_bytes: BIGINT
    ruta_almacenamiento: VARCHAR(512)
    fecha_carga: TIMESTAMP
    estado_procesamiento: VARCHAR(50)
}
```



```
entity EsquemasDeDatos {
  * esquema_id: UUID <<PK>>
  --
  # conjunto_datos_id: UUID <<FK>> <<UNIQUE>>
  fecha_creacion: TIMESTAMP
}
```

```
entity ColumnasDeEsquema {
  * columna_id: UUID <<PK>>
  --
  # esquema_id: UUID <<FK>>
  nombre_original: VARCHAR(255)
  nombre_mapeado: VARCHAR(255)
  tipo_dato: VARCHAR(50)
  es_objetivo: BOOLEAN
}
```

```
entity ModelosPredictivos {
  * modelo_id: UUID <<PK>>
  --
  # usuario_id: UUID <<FK>>
  # cuenta_id: UUID <<FK>>
  nombre: VARCHAR(255)
  descripcion: TEXT
  tipo_modelo: VARCHAR(50)
  parametros_config: JSONB
  estado: VARCHAR(50)
  fecha_creacion: TIMESTAMP
  fecha_ultimo_entrenamiento: TIMESTAMP
}
```

```
entity Predicciones {
  * prediccion_id: UUID <<PK>>
  --
  # modelo_id: UUID <<FK>>
  # conjunto_datos_id: UUID <<FK>>
  fecha_generacion: TIMESTAMP
  resultados_raw: JSONB
  metricas_precision: JSONB
}
```

```
entity Reportes {
```

```
* reporte_id: UUID <<PK>>
--
# usuario_id: UUID <<FK>>
# cuenta_id: UUID <<FK>>
nombre: VARCHAR(255)
tipo_grafico: VARCHAR(50)
configuracion_visual: JSONB
fecha_creacion: TIMESTAMP
fecha_ultima_modificacion: TIMESTAMP
}
```

```
entity Alertas {
  * alerta_id: UUID <<PK>>
  --
  # usuario_id: UUID <<FK>>
  # cuenta_id: UUID <<FK>>
  # modelo_id: UUID <<FK>>
  metrica_objetivo: VARCHAR(255)
  condicion: VARCHAR(50)
  umbral: DECIMAL(18,2)
  activa: BOOLEAN
  fecha_creacion: TIMESTAMP
  ultima_notificacion: TIMESTAMP
}
```

```
Cuentas |└-o{ Usuarios : "1 a muchos"
Cuentas |└-o{ ConjuntosDeDatos : "1 a muchos"
Cuentas |└-o{ ModelosPredictivos : "1 a muchos"
Cuentas |└-o{ Reportes : "1 a muchos"
Cuentas |└-o{ Alertas : "1 a muchos"
```

```
Usuarios |└-o{ ConjuntosDeDatos : "1 a muchos"
Usuarios |└-o{ ModelosPredictivos : "1 a muchos"
Usuarios |└-o{ Reportes : "1 a muchos"
Usuarios |└-o{ Alertas : "1 a muchos"
```

```
ConjuntosDeDatos |└-|{ EsquemasDeDatos : "1 a 1"
EsquemasDeDatos |└-o{ ColumnasDeEsquema : "1 a muchos"
```

```
ModelosPredictivos |└-o{ Predicciones : "1 a muchos"
ConjuntosDeDatos |└-o{ Predicciones : "1 a muchos"
```

```
Predicciones ||--o{ Reportes : "1 a muchos"  
Predicciones ||--o{ Alertas : "1 a muchos"
```

@enduml

7. Conclusiones

El diseño de base de datos propuesto para DataForge IA proporciona una estructura robusta y escalable para gestionar la información de usuarios, cuentas, conjuntos de datos, modelos predictivos, predicciones, reportes y alertas. La aplicación de principios de normalización asegura la integridad de los datos y minimiza la redundancia, mientras que el uso de UUIDs y tipos de datos JSONB ofrece flexibilidad y adaptabilidad a las necesidades cambiantes del análisis predictivo. Este diseño servirá como un pilar fundamental para la implementación del backend del sistema, garantizando que la información esté organizada de manera eficiente para soportar las funcionalidades de la plataforma.